2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC) | 979-8-3503-9354-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/ASP-DAC58780.2024.10473865

Sparse-Sparse Matrix Multiplication Accelerator on FPGA featuring Distribute-Merge Product Dataflow

Yuta Nagahara, Jiale Yan, Kazushi Kawamura, Masato Motomura, Thiem Van Chu
Tokyo Institute of Technology, Yokohama, Japan.
Email: {nagahara.yuta, yan.jiale, kawamura, motomura, thiem}@artic.iir.titech.ac.jp

Abstract— Sparse-Sparse matrix multiplication (SpMSpM) is a critical computation in various fields such as computational science and graph analysis. It poses computational challenges for general-purpose CPUs and GPUs due to its requirements for random memory access and the inherently low spatial/temporal locality. Given the increasing importance of SpMSpM, numerous accelerators have been recently proposed. However, they suffer from various issues such as low input utilization, heavy computational load, and excessive memory traffic during the merging process of intermediate results. This paper introduces a novel Distribute-Merge Product (DMP) SpMSpM dataflow and a DMP-based SpMSpM Architecture (DMSA). DMP distributes the workload into balanced streams, generates partial matrices based on these streams, and merges the partial results in a parallel and pipelined fashion. We have designed DMSA as a highly scalable architecture, implemented it on a Xilinx ZCU106 Evaluation Kit, and evaluated it on a set of benchmarks from the SuiteSparse matrix collection. When compared to a latest SpMSpM accelerator with approximately the same amount of hardware resources on the same FPGA platform, DMSA achieves 2.72× speedup, by facilitating the parallelism of partial matrix generation and merging. The speedup on the same platform reaches 4.80× when the parallelism explored in the merging process is doubled, evidencing the DMSA's superb scalability.

I. Introduction

Sparse-Sparse matrix Multiplication (SpMSpM) is an important computational kernel that is widely used in various fields, including theoretical chemistry [1], computational fluid dynamics [2, 3], electromagnetics [4, 5], graph analysis [6, 7, 8], and many other applications [9, 10, 11, 12]. Since a sparse matrix has plenty of zero elements, it is often stored in a compressed form to optimize memory usage. However, this results in irregular memory access and poor parallel processing patterns, which general-purpose CPUs and GPUs struggle to handle. Therefore, numerous domain-specific accelerators [13, 14, 15, 16, 17, 18, 19, 20] have been proposed for efficient SpMSpM processing.

Existing SpMSpM accelerators can be categorized into three types based on their dataflows: (1) Inner Product (IP) based [13], (2) Outer Product (OP) based [14, 15], and (3) Gustavson's Product (GP)[1] based [16, 17, 18, 19, 20]. (1) IP necessitates repetitive reading of one of the input matrices and performs the multiplication only on elements with matching indices. This results in a substantial memory read access overhead with a significant number of unused elements, especially when the density of the input matrices is low. (2) OP requires reading both the input matrices only one time and is free from the index matching issue. However, OP-based accelerators like OuterSPACE [14] face difficulties during the merging phase, as the size of the intermediate results increases proportionally with the dimensions of the input matrices. (3) GP-based accelerators address IP's issues by avoiding index matching and resolve OP's issues by computing the output matrix row-by-row, rather than generating numerous partial matrices and subsequently merging them. However, the variability in the number of nonzero elements from row to row in the input matrices makes it difficult to effectively and efficiently parallelize the computation due to the unbalanced workload required to compute each row of the output matrix. Another challenge is the irregular and random access pattern to one of the input matrices.

This paper proposes a new SpMSpM dataflow named DMP (Distribute-Merge Product). DMP retains the advantages of GP and addresses its load-balancing problem by distributing the workload in a way that the computation of the output matrix is transformed into a process of computing and merging multiple partial matrices in parallel. Building upon DMP, we develop a high-performance and highly efficient SpMSpM architecture named DMSA (DMP-based SpMSpM Architecture). Within DMSA, we design an input data loader to address the challenge of irregular and random access to an input matrix–an issue similar to that faced by GP. Furthermore, we architect an efficient mechanism for the parallel and pipelined computation and merging of multiple partial matrices. Contributions of this paper are as follows:

- We analyze the features of existing SpMSpM accelerators, and propose a new SpMSpM dataflow named DMP, which retains the advantages of GP and addresses its load-balancing problem.
- We leverage DMP to design DMSA, a high-performance and highly efficient SpMSpM architecture featuring a meticulously designed input data loader and a method of computing and merging multiple partial matrices in a parallel and pipelined fashion.

[1] It is also called row/column-wise product in previous studies.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on February 11,2026 at 05:06:23 UTC from IEEE Xplore. Restrictions apply.

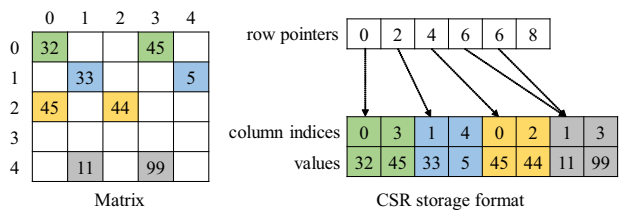

Fig. 1. CSR format: two arrays are used to represent the data: one containing column index/value pairs and the other consisting of row pointers.

- We optimize the DMSA's hardware resource utilization and implement it on a Xilinx ZCU106 platform.
- We evaluate DMSA on the Xilinx ZCU106 platform by using ten benchmark matrices from the SuiteSparse matrix collection [21]. Compared to Li *et al.* [20], a latest GP-based SpMSpM architecture that employs a similar volume of hardware resources on the same platform, DMSA achieves 2.72× speedup. When the parallelism degree of the architecture is doubled, the speedup further escalates to 4.80×.

II. Background and Related Work

Sparse matrices are commonly stored in a compressed format, where only nonzero elements are explicitly represented. One of the typical compression formats is Compressed Sparse Row (CSR), shown in Fig. 1. Although this type of compression method effectively reduces data volume, it introduces certain drawbacks, such as making random data access more challenging. As a result, it is important to design a dataflow that considers these disadvantages and develop a suitable architecture to handle them. Consequently, numerous SpMSpM accelerators have been proposed to address these challenges. In this section, we will classify existing accelerators into three primary categories based on their dataflows, and discuss their features and problems. For the convenience of explanation, in the rest of this paper, we refer to the left-hand side input matrix, the right-hand side input matrix, and the output matrix of the multiplication operation as $\boldsymbol{A}$, $\boldsymbol{B}$, and $\boldsymbol{C}$, respectively.

A. Inner Product (IP) based Accelerators

IP is a dataflow that computes the elements of Matrix $\boldsymbol{C}$ by computing the inner products of the row vectors of Matrix $\boldsymbol{A}$ and the column vectors of Matrix $\boldsymbol{B}$ (Fig. 2(a)). In this approach, unnecessary data read access is likely to occur. When computing the inner product of $\boldsymbol{A}(i,:)$ (the ith row of Matrix $\boldsymbol{A}$) and $\boldsymbol{B}(:,j)$ (the jth column of Matrix B), the intersection process is required to find k, where both $\boldsymbol{A}(i,k)$ and $\boldsymbol{B}(k,j)$ are nonzero elements. For sparse matrices, such k is often rare, leading to situations where one of the data elements is read but not utilized in multiplication.

One notable accelerator implementing this dataflow is SIGMA [13]. It arranges PEs (Processing Elements) in a systolic array for parallel inner-product calculations. However, with extremely sparse matrices, the number of executed multiplications is small relative to the number of data received, leading to inefficiencies in utilizing the PEs.

B. Outer Product (OP) based Accelerators

OP is a dataflow that computes Matrix $\boldsymbol{C}$ by summing all partial matrices derived from calculating the outer product of the column vectors of Matrix $\boldsymbol{A}$ and the row vectors of Matrix $\boldsymbol{B}$, as depicted in Fig. 2(b). OP's strengths lie in its elimination of the intersection process required in IP and its ease of parallelizing the multiplication process. However, it struggles when handling matrices with large dimensions. The size of each partial matrix is dictated by the height of Matrix $\boldsymbol{A}$ and the width of Matrix $\boldsymbol{B}$, with the number of these partial matrices matching the number of columns in Matrix $\boldsymbol{A}$. As the matrix size increases, the burden of merging these partial matrices intensifies.

An SpMSpM accelerator utilizing OP is OuterSPACE [14]. While OUTERSPACE benefits from the high parallelism of multiplication, it encounters challenges in merging partial matrices when the input matrices are large. SpArch [15] attempts to alleviate this issue by scheduling the merge process to reduce memory traffic, but its effectiveness is limited to only extremely sparse matrices.

C. Gustavson's Product (GP) based Accelerators

GP is a dataflow that computes Matrix $\boldsymbol{C}$ row-by-row[2]. It achieves this by adding all row vectors resulting from the scalar-vector products of nonzero elements in $\boldsymbol{A}$ and the corresponding row vectors of $\boldsymbol{B}$ (refer to Fig. 2(c)). Each row of Matrix $\boldsymbol{C}$, designated as $\boldsymbol{C}(i,:)$, is obtained by performing the scalar-vector product of $\boldsymbol{A}(i,k)$ and $\boldsymbol{B}(k,:)$ across all k values, followed by the aggregation of the resultant row vectors. Since GP performs scalar-vector multiplications, it eliminates the need for a coordinate intersection of nonzero elements. Moreover, even for large input matrices, the merging process is less burdensome than that in OP. This is due to GP's reliance on row-vector merging instead of the more cumbersome process of merging large partial matrices.

Capitalizing on these advantages, numerous GP-based accelerators have been introduced [16, 17, 18, 19, 20]. However, these accelerators suffer from two inherent problems of GP. The first problem revolves around the challenge of effectively and efficiently parallelizing the computation. This arises from the unbalanced workload associated with computing each row of Matrix $\boldsymbol{C}$, attributable to the variability in the number of nonzero elements across rows in Matrix $\boldsymbol{A}$ and Matrix $\boldsymbol{B}$. The second problem is the irregular and random access pattern

[2]Computing Matrix $\boldsymbol{C}$ row-by-row (row-wise) is one of the two versions of GP. The other version computes Matrix $\boldsymbol{C}$ column-by-column (column-wise). In this paper, for simplicity of presentation, we focus on only the row-wise version.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on February 11,2026 at 05:06:23 UTC from IEEE Xplore. Restrictions apply.

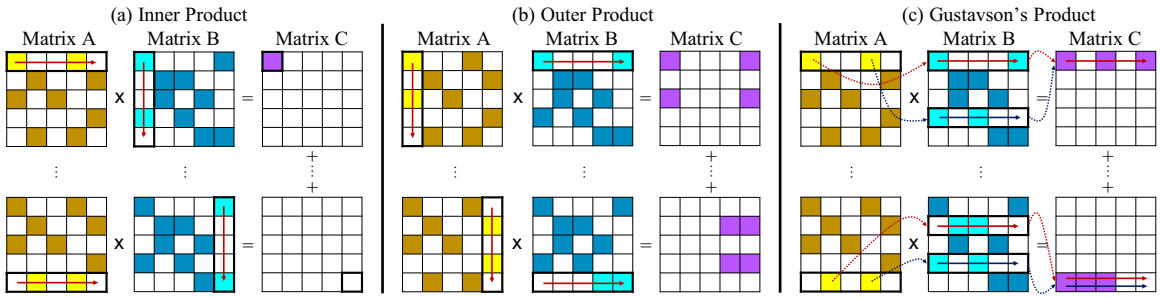

Fig. 2. Three existing SpMSpM dataflows: (a) Inner Product multiplies the row vectors of Matrix $\boldsymbol{A}$ and the column vectors of Matrix $\boldsymbol{B}$, (b) Outer Product multiplies the column vectors of Matrix $\boldsymbol{A}$ and the row vectors of Matrix $\boldsymbol{B}$, (c) Gustavson's Product computes row $\boldsymbol{C}(i,:)$ of Matrix $\boldsymbol{C}$ by adding all row vectors resulting from the scalar-vector product of $\boldsymbol{A}(i,k)$ and $\boldsymbol{B}(k,:)$ across all k values.

to Matrix $\boldsymbol{B}$. As illustrated in Fig. 2(c), a row from Matrix $\boldsymbol{B}$ may be accessed randomly multiple times depending on the distribution of nonzero elements in Matrix $\boldsymbol{A}$.

III. Proposed Dataflow and Architecture

A. Proposed Dataflow (DMP)

We propose a new SpMSpM dataflow called DMP, which retains the advantages of GP and makes it easier to effectively and efficiently parallelize the computation of Matrix $\boldsymbol{C}$ by transforming it into a process of computing and merging multiple partial matrices in parallel.

The mechanism of DMP is shown in Fig. 3. First, setting the number of partial matrices to be derived as p, the elements of $\boldsymbol{A}$ are distributed into p streams so that the ith nonzero element stored in the CSR format belongs to stream ($i \ mod \ p$). Next, the process of calculating a partial matrix in each stream is conducted in parallel. Specifically, for each stream, the scalar-vector products of the elements of $\boldsymbol{A}$ in this stream and the rows with the corresponding indices in $\boldsymbol{B}$ are conducted, and the results are merged and concatenated to generate the partial matrices. Finally, all the generated p partial matrices are merged to obtain the final result.

Compared to existing dataflows, DMP has the following three benefits. Firstly, the index matching to decide whether to multiply a nonzero element in $\boldsymbol{A}$ with one in $\boldsymbol{B}$ like in IP is not required. This avoids the issue of loading a nonzero element only for it not to be used. Secondly, unlike OP, the number of partial matrices (p) is a design parameter, and it is not tied to the dimensions of $\boldsymbol{A}$ and $\boldsymbol{B}$. This allows all partial matrices to be computed and merged in parallel to yield the final result, eliminating the need to store/load partial matrices to/from external memory. Thirdly, by evenly distributing the nonzero elements of Matrix $\boldsymbol{A}$ across p streams, DMP facilitates the effective and efficient parallelization of Matrix $\boldsymbol{C}$'s computation. In contrast to GP, DMP avoids the unbalanced workload challenge associated with computing each row of Matrix $\boldsymbol{C}$.

B. Proposed Architecture (DMSA)

We design DMSA (Fig. 4), an efficient SpMSpM accelerator architecture exploiting the advantages of DMP.

First, we briefly describe DMSA through its execution mechanism. The processing of an SpMSpM operation is kicked off when the *DRAM Interface* starts streaming the row pointers, column indices, and values of Matrix $\boldsymbol{A}$, which are stored in the CSR format in DRAM, to the *MatA Distributor*. The MatA Distributor converts the row pointers to row indices and distributes the (row index, column index, value) elements to p *Partial Matrix Compute Units (PMCUs)* according to the DMP dataflow (Fig. 5(a)). Based on the elements from the MatA Distributor, the PMCUs send requests to the *MatB Loader* to get necessary Matrix $\boldsymbol{B}$ data, calculate the partial matrices, and stream the results to the *Merge Tree* (Fig. 5(b) and Fig. 6). Finally, the merged result of the partial matrices is streamed to the *Post-Processing Unit* where elements with the same row index and column index are accumulated to produce the result matrix in the CSR format.

Next, we present the major challenges encountered when designing DMSA and our solutions for them. The first challenge is how to access Matrix $\boldsymbol{B}$ efficiently. Like in the GP dataflow, the access pattern to Matrix $\boldsymbol{B}$ in DMP is irregular and depends on the pattern of nonzero elements in Matrix $\boldsymbol{A}$. The challenge is further complicated by the fact that each access to Matrix $\boldsymbol{B}$ has a different data length because the unit of access is the effective elements in a row. The second challenge is how to compute the partial matrices and merge them in a way that provides a high throughput without requiring excessive hardware resources. We will describe our approaches to these challenges in Sections B.1 and B.2 below.

B.1 Matrix B Loading Mechanism

In DMP, each nonzero element $\boldsymbol{A}(i,j)$ of Matrix $\boldsymbol{A}$ is multiplied with row $\boldsymbol{B}(j,:)$ of Matrix $\boldsymbol{B}$. Since the nonzero elements of Matrix $\boldsymbol{A}$ are loaded and processed in the order of the CSR format, there is no regularity in the pattern of their column indices. This results in an irregular pattern of rows of Matrix $\boldsymbol{B}$ that need to be loaded; and each row may be accessed

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on February 11,2026 at 05:06:23 UTC from IEEE Xplore. Restrictions apply.

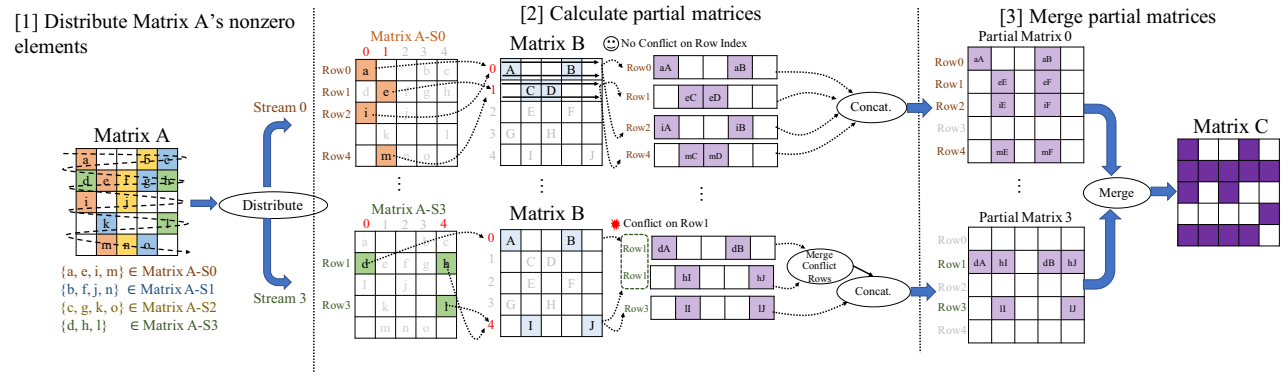

Fig. 3. Overview of DMP. In the initial phase, the nonzero elements of Matrix A are accessed in the CSR order and distributed into p ($p = 4$ case is explained in this figure) distinct streams. Subsequently, in each stream, an element of Matrix A with a column index of i is multiplied by the ith row vector of Matrix B. Intermediate results sharing the same row index are merged before concatenated. Ultimately, all the partial matrices are merged to produce Matrix C.

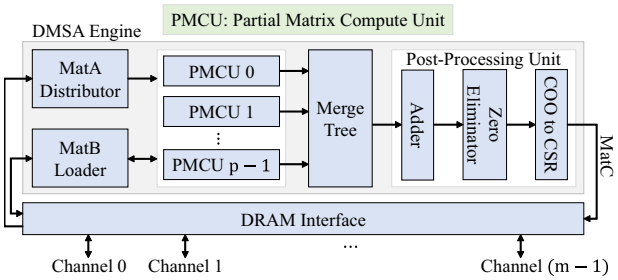

Fig. 4. Overview of DMSA, our proposed SpMSpM accelerator architecture based on DMP dataflow.

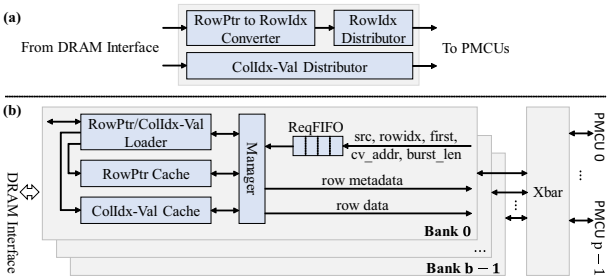

Fig. 5. (a) MatA Distributor; (b) MatB Loader.

multiple times. This issue is similar to that in the GP dataflow, though the access pattern to Matrix $\boldsymbol{B}$ is not the same.

Another issue that needs to be addressed is the unpredictability of row lengths. When a PMCU issues a request for loading a row, say r, of Matrix $\boldsymbol{B}$ to the MatB Loader, the ideal scenario is that the MatB Loader can fetch all data of r and send them back to the PMCU without any interruption. However, this is hard to achieve since the length of r is unknown at the request time. A potential solution is to employ a large buffer, which can store the whole length of the longest row at each request source. However, it is costly and may decrease the overall performance since a request can be issued only when there is enough buffer space for the longest row.

Fig. 5(b) shows our MatB Loader design. For serving multiple requests from the PMCUs at the same time, we adopt a multi-bank architecture where bank t is responsible for loading rows r with $r\%b = t$. Here, b is the number of banks. It is set based on the number of DRAM channels available on the target platform. A crossbar is used to connect requests from the PMCUs to their corresponding banks. In each bank, we store the incoming requests in a FIFO (*ReqFIFO*), so that new requests can be accepted while a request is being served.

Inspired by previous GP dataflow studies [17, 19, 20], which uncover that the access pattern to Matrix $\boldsymbol{B}$, despite being irregular, exhibits considerable locality, we design a caching mechanism for Matrix $\boldsymbol{B}$ loading. To deal with unpredictable row lengths, it is designed to allow multiple requests for a single row depending on the length of that row and the free buffer space at the request source. A request from a PMCU to the MatB Loader consists of five pieces of information: (1) request source (`src`), (2) index of the row to be loaded (`rowidx`), (3) a 1-bit flag determining whether this is the first request for loading this row (`first`), (4) start address of the remaining data if this is the second or successive request (`cv_addr`), and (5) the maximum number of cache blocks that the request source can accept in case `first` = 1 or the exact number of requested cache blocks in case `first` = 0 (`burst_len`).

The MatB Loader serves each request as follows. First, the *Manager* examines the `first` flag to determine the request type. If this is the first request for the target row (i.e. `first` = 1), the manager will check the *RowPtr Cache* for two row pointers at indices `rowidx` and `rowidx` + 1. If a cache miss occurs, the manager will ask the *RowPtr/ColIdx-Val Loader* to issue a load request to the DRAM interface. Next, based on the row pointers, the manager calculates the metadata necessary to access the column index and value data of the target row, sends this metadata to the request source, and at the same

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on February 11,2026 at 05:06:23 UTC from IEEE Xplore. Restrictions apply.

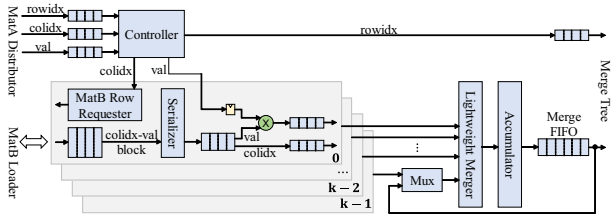

Fig. 6. PMCU architecture.

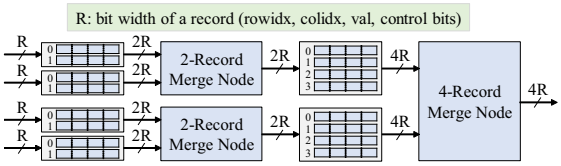

Fig. 7. A merge tree that can merge four sorted input sequences simultaneously and output four records per cycle.

time accesses the *ColIdx-Val Cache*. Similar to when accessing the RowPtr Cache, on a cache miss, the RowPtr/ColIdx-Val Loader will be asked to issue a load request to the DRAM interface. By using the metadata provided by the MatB Loader, the request source can determine whether it is necessary to issue another request for the remaining data of the current row and its start address. In this way, for any row, the MatB Loader only has to access the row pointers in the first request.

B.2 Partial Matrix Computing and Merging Mechanism

Fig. 6 shows our architecture for computing a partial matrix. There are k multipliers managed by a controller. When there are k or less elements from the Matrix $\boldsymbol{A}$ distributor having the same row index, the corresponding row of the partial matrix is obtained by only one round of multiplying and merging based on the column index. On the other hand, when there are more than k elements with the same row index, multiple rounds of multiplying and merging are required. From the second round, the multiplication results are merged with the result of the previous round, which is fed back from the *Merge FIFO* to the last input of the *Lightweight Merger*. As the name suggests, the lightweight merger is a simple circuit that outputs the (column index, value) pair with the smallest column index among k inputs. We choose to use a simple merger here to be able to increase k for reducing the number of multiplying and merging rounds. Finally, to reduce the size of the Merge FIFO and the load of the post-processing unit, we place an accumulator at the output of the lightweight merger to accumulate the values of the same column index.

Our design requires that the Merge FIFO is large enough to store all data that will need to be fed back in any round of multiplying and merging. Fortunately, our analysis of a wide variety of benchmark matrices from SuiteSparse [21] shows that the required size is not large and can be fit into FPGA on-chip memory easily. For the benchmark matrices used in this paper, a 5KB Merge FIFO is enough for both configurations in the evaluation section.

With the design described above, each PMCU can achieve a throughput up to one record (consisting of row index, column index, value, and three control bits) per cycle. With p PMCUs running in parallel, the aggregate throughput is up to p records per cycle. To fully exploit this, we have designed a merge tree capable of accepting p input streams, each with a throughput of one record per cycle, and outputting p records per cycle (Fig. 7; $p = 4$ in this figure), leveraging the principles derived from recent foundational studies on hardware merge sorters [22, 23, 24, 25]. Our design of the mechanism for merging two data streams within each merge node is especially inspired by FLiMS [23]. It allows for the customization of the number of pipeline stages to meet specific hardware resource and operating frequency requirements. Furthermore, we have developed a credit-based flow control scheme that effectively precludes stalls in the merging pipeline, which can result from the absence of data in one or both input streams, an issue arising primarily due to two factors: the inability of the MatA Distributor and the MatB Loader to match the processing speed of the PMCUs because of the DRAM access overhead, and the accumulation of data with identical indices in the PMCUs.

IV. Evaluation

A. Evaluation Setup

The evaluation is conducted on a Xilinx ZCU106 Evaluation Kit, featuring a Zynq UltraScale+ XCZU7EV-2FFVC1156 MPSoC. The DMSA engine is developed using Verilog while the DRAM interface is written in C++ and converted to Verilog using Vitis HLS. We use Vitis and Vivado 2022.2 for system integration and bitstream generation.

We compare DMSA with the GP-based SpMSpM architecture proposed by Li *et al.* [19, 20], which was also evaluated on a Xilinx ZCU106 Evaluation Kit. DMSA can run at 185 MHz, but we intentionally reduced the operating frequency to 100 MHz, the frequency of the previous work [20], in order to highlight the efficiency of DMP and DMSA. The overall hardware resource consumption of our designs, DMSA(4, 8) and DMSA(8, 4), is compared with that of [20], as demonstrated in Table II. Here, DMSA(p, k) denotes a DMSA with p PMCUs and k multipliers per PMCU. It is noteworthy that DMSA(4, 8) consumed approximately the same amount of hardware resources as [20], while DMSA(8, 4) required more of them.

This study uses the same ten benchmark matrices from the SuiteSparse matrix collection [21] as in [20]. As in all previous studies, we evaluate the SpMSpM operation of multiplying a sparse matrix by itself. The data type of values is 32-bit float, and both column indices and row pointers are 32-bit unsigned integers. Two input matrices are stored in the CSR format without any additional preprocessing and the output matrix is also written back to DRAM in the CSR format.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on February 11,2026 at 05:06:23 UTC from IEEE Xplore. Restrictions apply.

TABLE I
BENCHMARK MATRICES FROM THE SUITESPARSE MATRIX COLLECTION

Benchmark	#row(=#col)	nnz	MatC's nnz	Number of execution cycles		
				Li *et al.* [20]	DMSA(4, 8)	DMSA(8, 4)
poisson3Da	13,514	352,762	2,957,530	16,638,816	5,303,369	3,359,405
raefsky1	3,342	294,276	1,184,340	25,658,832	9,881,110	6,978,575
crystk01	4,875	315,891	1,200,591	20,202,582	7,584,840	4,758,675
s3rmt3m3	5,357	207,695	545,441	8,411,161	3,207,456	1,590,736
t2dah_a	11,445	176,117	613,497	5,354,111	1,927,486	956,002
nasa2910	2,910	174,296	890,750	11,731,798	4,874,951	3,102,054
bcsstk24	3,562	159,910	446,474	7,354,144	2,703,064	1,373,522
cavity26	4,562	138,187	613,291	6,012,791	2,266,696	1,382,999
ex9	3,363	99,471	373,815	3,790,519	1,354,922	685,930
af23560	23,560	484,256	2,338,788	13,979,280	4,819,497	2,411,311

nnz denotes the number of nonzero elements.
DMSA(p, k) denotes a DMSA with p PMCUs and k multipliers per PMCU.

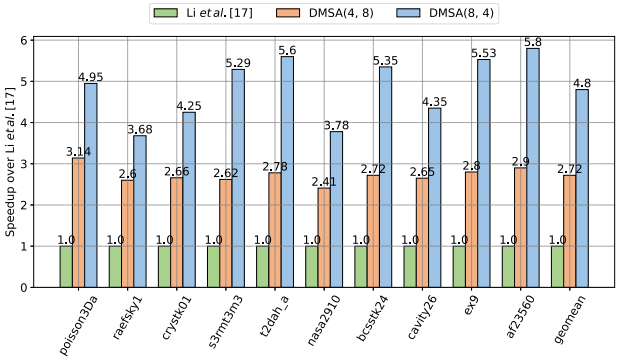

Fig. 8. Speedup of DMSA over the GP-based SpMSpM architecture proposed in [20].

TABLE II
HARDWARE RESOURCE UTILIZATION ON ZCU106

Architecture		LUTs	FFs	BRAMs	URAMs	DSPs
Li *et al.* [20]		66.29%	42.11%	64.42%	62.50%	2.66%
DMSA(4, 8)	Total	61.82%	26.50%	71.15%	66.67%	6.60%
	DMSA Engine	48.31%	12.54%	21.79%	66.67%	6.60%
	DRAM Interface	6.43%	6.15%	15.38%	0.00%	0.00%
	Platform	7.08%	7.81%	33.97%	0.00%	0.00%
DMSA(8, 4)	Total	83.29%	38.55%	79.49%	66.67%	11.23%
	DMSA Engine	69.10%	23.82%	24.36%	66.67%	11.23%
	DRAM Interface	6.62%	6.57%	17.31%	0.00%	0.00%
	Platform	7.57%	8.16%	37.82%	0.00%	0.00%

DMSA(p, k) denotes a DMSA with p PMCUs and k multipliers per PMCU.

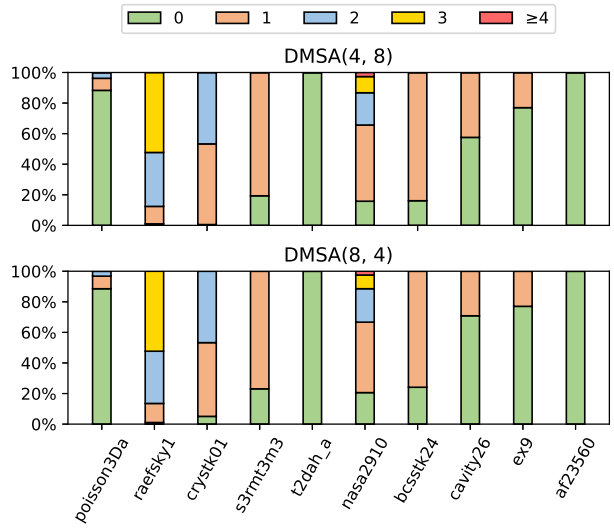

Fig. 9. Breakdown of the number of required feedback rounds for each benchmark matrix.

B. Evaluation Results

In order to conduct a fair performance comparison between our design and [20], and to clarify the speedup achieved by the proposed dataflow and architecture, we have simulated the exact cycle count of the whole SpMSpM process from the initial phase of input matrix readout to the final phase of output matrix writing back. The simulation has been conducted using Synopsys VCS. The results are shown in the three rightmost columns of Table I and illustrated in Fig. 8. We can see that the average speedup for DMSA(4, 8) is 2.72×, and DMSA(8, 4) achieves a notable average speedup of 4.80×.

The speedup achieved by DMSA can be attributed to the improved parallelism during the computation and merging of partial matrices, as opposed to the row-by-row computation of the output matrix in [20]. DMSA(p, k) efficiently computes all p partial matrices in parallel, which does not require any inter-process synchronization. Moreover, during these computations, the feedback of multiplication results is required only when a row of Matrix $\boldsymbol{A}$ contains more than $p * k$ nonzero elements ($p * k = 32$ in all of our experiments). In contrast, in the SpMSpM architecture in [20], the computation of Matrix $\boldsymbol{C}$ is in a row-by-row fashion and the feedback is required when a row of Matrix $\boldsymbol{A}$ contains more than four nonzero elements. This difference is one of the main reasons that make DMSA perform better.

Fig. 9 shows a detailed breakdown of the number of required feedback rounds for each benchmark matrix. Notably, for three benchmark matrices *poisson3Da*, *t2dah_a*, and *af2350*, no feedback is required (i.e., the number of feedback rounds is zero) for approximately 90% or more of the rows of Matrix A. This contributes to above-average speedups for these benchmarks. Conversely, the matrices *raefsky1* and *nasa2910* exhibit lower speedups, attributable to the need for three or more feedback rounds for a substantial fraction of the rows of Matrix $\boldsymbol{A}$.

An additional factor contributing to DMSA's superior speedup over the SpMSpM architecture presented in [20] is the difference in the throughput of merging partial results (partial matrices in DMSA and partial rows in [20]). Specifically, the DMSA(p, k) design is capable of outputting up to p elements per cycle, whereas the merging unit in [20] is limited to producing only a single element per cycle.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on February 11,2026 at 05:06:23 UTC from IEEE Xplore. Restrictions apply.

V. CONCLUSION

Based on a comprehensive analysis of existing IP-, OP-, and GP-based accelerators, we propose a new SpMSpM dataflow called DMP. DMP retains the advantages of GP over IP and OP, and makes it easier to effectively and efficiently parallelize the computation by addressing the load-balancing problem. We use DMP to develop DMSA, an SpMSpM architecture featuring an efficient input data loader and a unique mechanism for parallel and pipelined computation and merging of multiple partial matrices. We implement this architecture on a Xilinx ZCU106 platform and evaluate its performance using benchmarks from the SuiteSparse matrix collection. The results indicate that DMSA outperforms a latest GP-based SpMSpM architecture implemented on the same FPGA platform, achieving an average speedup of 2.72× with comparable hardware resources. Moreover, by doubling the parallelism within the merging process, the speedup surges to 4.80×, highlighting DMSA's exceptional scalability.

ACKNOWLEDGMENTS

This work was partially supported by KAKENHI Grant Numbers 23H05489 and 21K17720, Japan.

REFERENCES

[1] Itoh et al., "Order-N Tight-Binding Molecular Dynamics on Parallel Computers," *Computer Physics Communications*, vol. 88, no. 2-3, pp. 173–185, 1995.

[2] Oyarzun et al., "An FPGA Cached Sparse Matrix Vector Product (SpMV) for Unstructured Computational Fluid Dynamics Simulations," *arXiv*, 2021.

[3] Kashi and Nadarajah, "Fine-grain Parallel Smoothing by Asynchronous Iterations and Incomplete Sparse Approximate Inverses for Computational Fluid Dynamics," in *AIAA Scitech Forum*, 2020.

[4] Alléon et al., "Sparse Approximate Inverse Preconditioning for Dense Linear Systems Arising in Computational Electromagnetics," *Numerical Algorithms*, vol. 16, pp. 1–15, 1997.

[5] Lee et al., "Sparse Inverse Preconditioning of Multilevel Fast Multipole Algorithm for Hybrid Integral Equations in Electromagnetics," *IEEE Transactions on Antennas and Propagation*, vol. 52, no. 9, pp. 2277–2287, 2004.

[6] Kipf and Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *ICLR*, 2017.

[7] Petar et al., "Graph Attention Networks," in *ICLR*, 2018.

[8] Mattson et al., "Standards for Graph Algorithm Primitives," in *HPEC*, 2013, pp. 1–2.

[9] Gilbert et al., "A Unified Framework for Numerical and Combinatorial Computing," *Computing in Science & Engineering*, vol. 10, no. 2, pp. 20–25, 2008.

[10] Penn, "Efficient Transitive Closure of Sparse Matrices over Closed Semirings," *Theoretical Computer Science*, vol. 354, no. 1, pp. 72–81, 2006.

[11] Chan, "More Algorithms for All-Pairs Shortest Paths in Weighted Graphs," in *STOC*, 2007, pp. 590–598.

[12] Yamazaki and Li, "On Techniques to Improve Robustness and Scalability of a Parallel Hybrid Linear Solver," in *VECPAR*, 2010, pp. 421–434.

[13] Qin et al., "SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training," in *HPCA*, 2020, pp. 58–70.

[14] Pal et al., "OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator," in *HPCA*, 2018, pp. 724–736.

[15] Zhang et al., "SpArch: Efficient Architecture for Sparse Matrix Multiplication," in *HPCA*, 2020, pp. 261–274.

[16] Srivastava et al., "MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product," in *MICRO*, 2020, pp. 766–780.

[17] Zhang et al., "Gamma: Leveraging Gustavson's algorithm to accelerate sparse matrix multiplication," in *ASPLOS*, 2021, pp. 687–701.

[18] Baek et al., "InnerSP: A Memory Efficient Sparse Matrix Multiplication Accelerator with Locality-Aware Inner Product Processing," in *PACT*, 2021, pp. 116–128.

[19] Li and Liu, "Accelerating Gustavson-based SpMM on Embedded FPGAs with Element-wise Parallelism and Access Pattern-aware Caches," in *DATE*, 2023, pp. 1–6.

[20] Li et al., "An Efficient Gustavson-based Sparse Matrix-matrix Multiplication Accelerator on Embedded FPGAs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.

[21] Davis and Hu, "The University of Florida Sparse Matrix Collection," *ACM TOMS*, vol. 38, no. 1, pp. 1–25, 2011.

[22] Song et al., "Parallel Hardware Merge Sorter," in *FCCM*, 2016, pp. 95–102.

[23] Papaphilippou et al., "FLiMS: A Fast Lightweight 2-Way Merger for Sorting," *IEEE Transactions on Computers*, vol. 71, no. 12, pp. 3215–3226, 2022.

[24] Saitoh et al., "A High-Performance and Cost-Effective Hardware Merge Sorter without Feedback Datapath," in *FCCM*, 2018, pp. 197–204.

[25] Mashimo et al., "High-Performance Hardware Merge Sorter," in *FCCM*, 2017, pp. 1–8.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR. Downloaded on February 11,2026 at 05:06:23 UTC from IEEE Xplore. Restrictions apply.